Automated Network Management System on Red Hat 9

Table of Contents

1. ABSTRACT:

In the modern IT infrastructure landscape, automation plays a pivotal role in achieving speed, consistency, and reliability in system administration tasks. This project focuses on the implementation of Ansible—a powerful open-source automation tool—for automating critical system configurations across multiple Linux-based systems. Specifically, this project automates four core areas: network interface configuration using dummy interfaces for testing, DNS resolver management, firewall rule enforcement using iptables on RHEL 9, and network diagnostics using automated scripts.

The primary aim is to reduce manual administrative overhead, minimize configuration errors, and enhance system security through repeatable and reliable Ansible playbooks. The setup is tested in a virtualized environment using VMware, where one RHEL 9 system functions as the Ansible control node and multiple RHEL 8 clients are managed remotely over SSH.

Each task is modularized as a script and executed through a centralized Ansible playbook (main.yml). Comprehensive logging is implemented to capture the output of each task, providing transparency and aiding in debugging. The use of a dummy interface ensures safe testing of network configurations without disrupting active interfaces.

2. LIST OF FIGURES:

3. INTRODUCTION:

In today's rapidly evolving IT environments, system administrators are expected to manage complex infrastructure with speed, precision, and consistency. Traditional manual configuration methods are not only time-consuming but also error-prone, especially when applied across multiple machines. The rise of configuration management tools like **Ansible** has transformed how systems are deployed, maintained, and secured by introducing automation into system administration.

Ansible is an open-source, agentless automation tool developed by Red Hat, known for its simplicity, scalability, and efficiency. It uses **SSH** for secure communication and **YAML-based playbooks** to define the desired state of systems. It eliminates the need for manual intervention by automating repetitive tasks such as package installation, service management, configuration enforcement, and security hardening.

This project is a practical implementation of Ansible to automate four core administrative tasks:

1. **Interface Configuration** – Using dummy interfaces to simulate real network configurations for testing purposes.
2. **DNS Management** – Automating the modification of /etc/resolv.conf using a Python script to ensure consistent DNS settings.
3. **Firewall Setup** – Installing and configuring iptables on RHEL 9 systems to enforce secure firewall policies with port-specific allow and deny rules.
4. **Network Diagnostics** – Running diagnostic commands through a Python automation script to monitor network health and connectivity.

The complete solution is managed via a single **Ansible playbook (main.yml)**, which sequentially executes the scripts on target machines. These scripts are designed to be modular, maintainable, and extensible, allowing easy updates and scalability for larger environments.

By integrating logging for each task, the system administrator gains visibility into the execution flow and outcomes of each configuration step. The project not only showcases automation capabilities but also emphasizes security, reproducibility, and the power of Infrastructure as Code (IaC).

This project is particularly useful for:

- System administrators aiming to standardize configurations
- Educational environments simulating real-world DevOps practices

4. OBJECTIVE:

The primary objective of this project is to **design, develop, and implement an automated system configuration framework** using **Ansible** to simplify and secure essential administrative tasks in a Linux-based environment.
The specific objectives of the project are:

To Automate Network Interface Configuration

- Create and configure **dummy network interfaces** to simulate real-world scenarios without affecting live network interfaces.
- Ensure repeatability and error-free setup using shell scripts triggered by Ansible.

To Manage DNS Settings Automatically

- Modify and manage DNS configurations on multiple systems using a centralized **Python script** executed through an Ansible playbook.
- Standardize DNS resolution across all managed nodes by automating /etc/resolv.conf updates.

To Install and Configure Firewall Using Iptables on RHEL 9

- Automate the installation of iptables-legacy and setup of default security policies.
- Define and apply **allow and deny rules** for specific ports to ensure controlled network access.

To Perform Network Diagnostics Automatically

- Develop a Python-based script to run **diagnostic commands** like ping, ip a, and ip r remotely.
- Centralize execution via Ansible for consistent output and easier troubleshooting.

To Centralize Execution and Logging through Ansible

- Use a master playbook (main.yml) to sequence and control all tasks across multiple nodes.
- Maintain detailed **log files** for each operation to improve transparency, auditing, and debugging.

5. LITERATURE REVIEW:

In the domain of system administration and DevOps, automation tools have revolutionized infrastructure management by providing reliability, scalability, and efficiency. Over the past decade, multiple tools such as **Puppet**, **Chef**, **SaltStack**, and **Ansible** have emerged to implement **Infrastructure as Code (IaC)**. Among these, **Ansible** has gained wide adoption due to its simplicity, agentless architecture, and human-readable syntax.

➢ Evolution of Configuration Management Tools

Early methods of configuring systems involved manual intervention, custom scripts, or batch files. This was error-prone, unscalable, and inconsistent across systems. Tools like **Puppet** and **Chef** introduced declarative and procedural configuration management but required a steep learning curve and agent installations.

Ansible, introduced by Michael DeHaan in 2012 and later acquired by **Red Hat**, addressed many limitations of previous tools. It offered:

- **Agentless architecture** using SSH
- **Push-based deployment** model
- **YAML-based playbooks**, which are easy to read and write

These innovations made Ansible an ideal choice for organizations shifting towards **DevOps and Continuous Deployment (CD)** pipelines.

➢ Research on Ansible in Academic and Industry Contexts

Several studies and case reports have evaluated Ansible for infrastructure automation:

- **Sharma et al. (2018)** compared Ansible with Chef and Puppet and concluded that Ansible had faster deployment times and a simpler learning curve.
- **Kumar and Patel (2020)** highlighted Ansible's effectiveness in multi-node environments, particularly in network configuration automation using playbooks.
- A research paper from **IEEE Xplore (2021)** described how using Ansible improved system recovery time and reduced downtime in enterprise environments by 40%.
  In addition, organizations like **NASA**, **Accenture**, and **Autodesk** have reported success stories in implementing Ansible for large-scale IT operations.

➢ Use of Scripting for System Configuration

Shell scripts and Python scripts have long been used for automation. Shell scripting remains foundational for tasks like network setup and firewall configuration, while **Python is preferred for logic-heavy operations**, such as parsing and diagnostics.

This project leverages both shell and Python scripts—executed via Ansible—to create a hybrid automation framework.

➢ Gap Identified and Project Justification

While several works focus on service provisioning or DevOps pipelines, **this project uniquely combines network configuration, DNS automation, firewall management, and diagnostics** into a single Ansible-driven framework with complete logging. It aims to fill the gap in projects that holistically automate system-level configurations while keeping the setup lightweight and modular.

6. THEORY:

This section explores the theoretical foundation of the project, including the core concepts that make Ansible an efficient automation tool. Understanding the architecture, communication mechanism, configuration language, and automation principles is essential to fully appreciate how Ansible performs infrastructure automation.

6.1 Ansible Architecture

Ansible follows a **simple, agentless architecture**. It uses SSH to connect to managed nodes, eliminating the need to install agents on each client machine. Its architecture is based on the following components:

- **Control Node**: The system where Ansible is installed and from which commands are issued. It contains the playbooks, inventory file, and configuration settings.
- **Managed Nodes**: The target systems (e.g., RHEL 8 clients) that receive and execute the tasks from the control node.
- **Inventory**: A file (typically hosts or inventory.ini) listing all the managed nodes along with their connection details.
- **Modules**: Ansible uses hundreds of pre-built modules (e.g., apt, service, firewalld, copy) to perform tasks. Custom scripts can also be run using the script module.
- **Playbooks**: Written in YAML, playbooks define the tasks to be executed on the managed nodes. Each task maps to a module call.
- **Facts**: System information collected by Ansible from the managed nodes, used to make conditional decisions within tasks.

This architecture allows **centralized control** and **stateless operations**, making Ansible lightweight and highly scalable.

6.2 SSH-Based Automation

Ansible relies on **SSH (Secure Shell)** for secure and agentless communication between the control node and the managed nodes. SSH offers the following benefits in automation:

- **Encryption**: Ensures secure data transmission between nodes.
- **Authentication**: Uses password-based or key-based authentication. For automation, **SSH key pairs** are preferred.
- **No Agent Required**: Unlike tools like Puppet or Chef, Ansible does not require any daemon or agent to be running on the target node.
- **Portability**: Since SSH is natively supported in Unix/Linux systems, Ansible can connect to virtually any Linux-based host out-of-the-box.

In this project, SSH is used to:

- Push scripts (e.g., firewall-setup.sh, dns-manager.py) from the control node to the client systems.
- Execute them securely and collect logs/output in real time.

6.3 YAML Playbook Syntax

Ansible uses **YAML (YAML Ain't Markup Language)** for writing playbooks. YAML is a human-readable data serialization language that emphasizes clarity and simplicity. A basic Ansible playbook consists of:

- **Hosts**: The target nodes where tasks will be executed.
- **Tasks**: The list of operations to perform, such as installing a package, configuring a file, or restarting a service.
- **Modules**: Each task uses a module to interact with the system.
- **Variables**: Dynamic values used for customization and reusability.
- **Handlers**: Special tasks triggered upon changes (e.g., restart a service when configuration changes).

Example:

```
--
- name: Install Apache
  hosts: webservers
  become: yes
  tasks:
    - name: Install httpd
      yum:
        name: httpd
        state: present
```

The clarity of YAML makes it easier for administrators and developers to collaborate and maintain automation workflows.

6.4 Importance of Idempotency in Configuration Management

Idempotency is a key principle in configuration management. An operation is **idempotent** if performing it multiple times has the **same effect** as performing it once. This ensures:

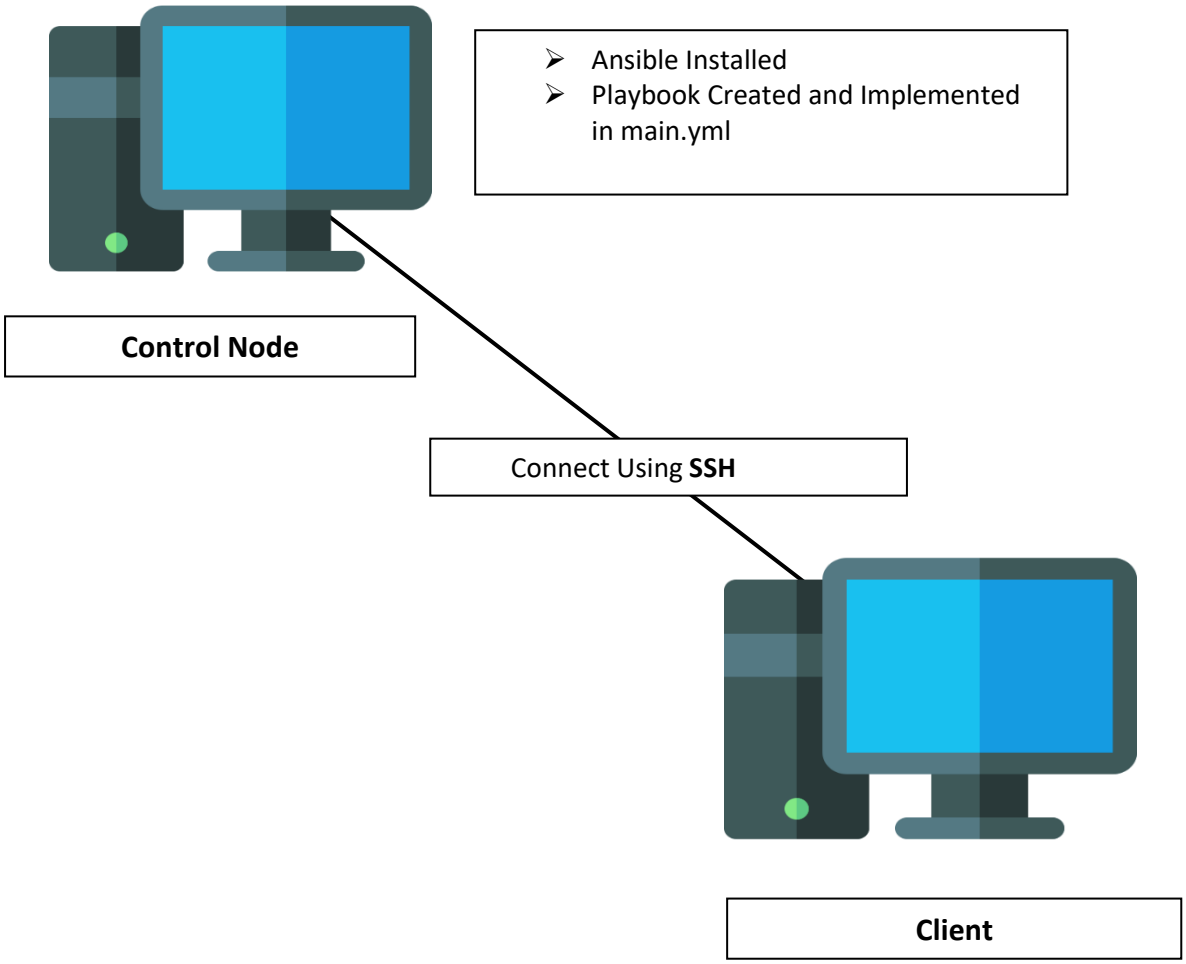
- **Safe Re-execution**: Tasks can be run repeatedly without risk of unwanted side effects.
- **State Assurance**: The system always ends up in the desired configuration state, regardless of its starting point.
- **Efficiency**: Ansible skips tasks that are already in the desired state, improving performance.
- **Error Reduction**: Reduces the chance of system drift or inconsistent configuration across environments.

In this project:

- Playbooks ensure that interfaces are created **only if not already present**.
- Firewall rules are reapplied **only if needed**.
- DNS configurations are enforced consistently across all managed nodes.

Thus, idempotency helps make Ansible-based automation **reliable**, **predictable**, and **repeatable**.

7. PROJECT DIAGRAM:

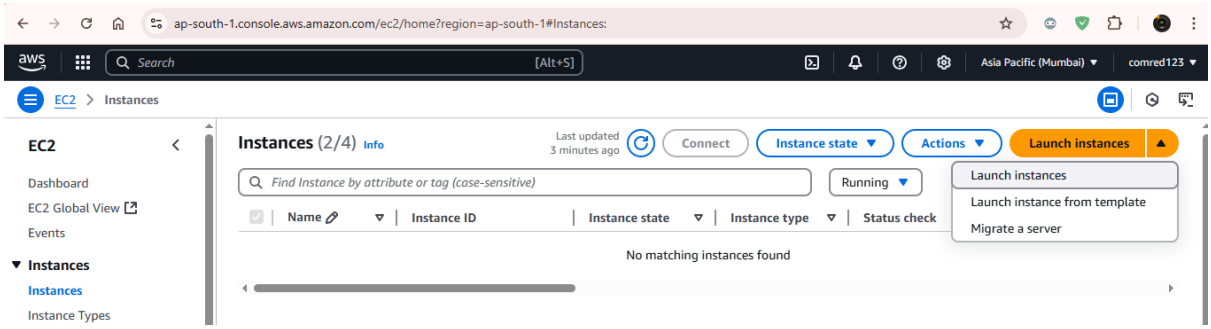


8. IMPLEMENTATION:

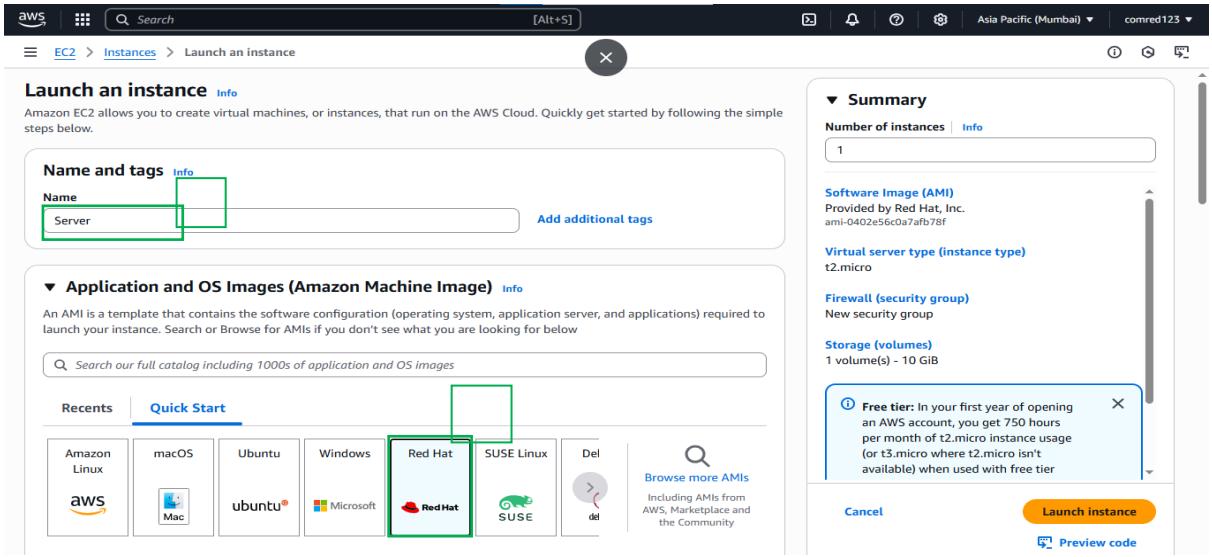
8.1 Environment Setup

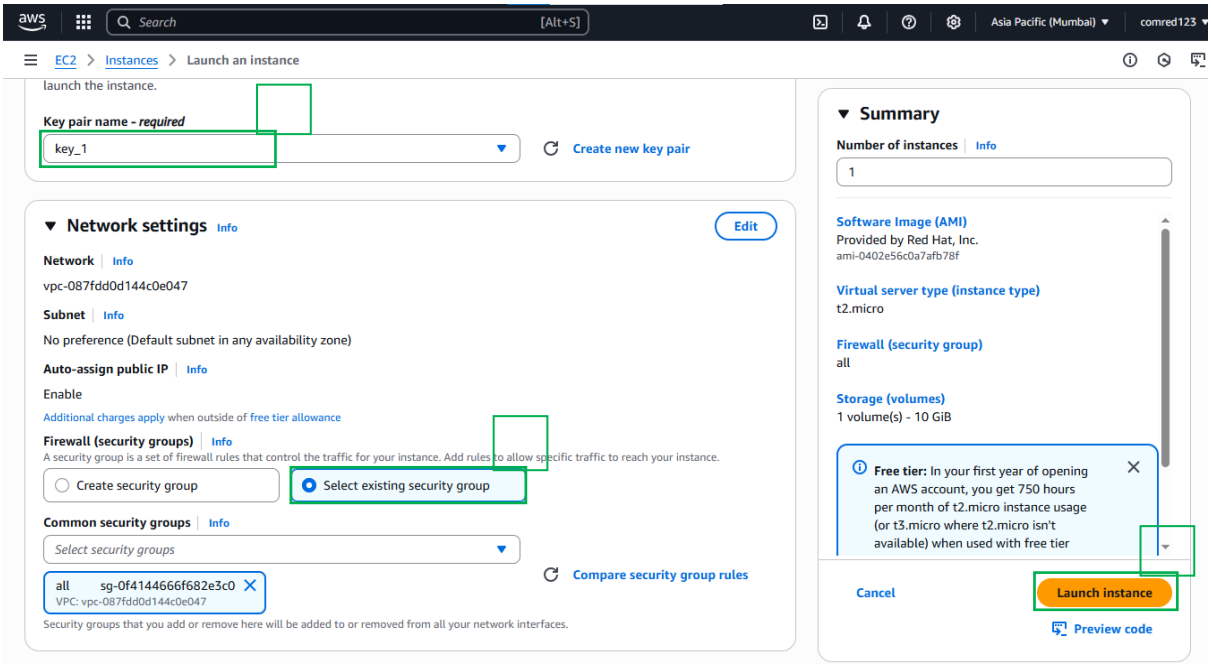
8.1.1 Creating EC2 Instances

- Login to **aws.amazon.com**
- Click on **Lunch instances**

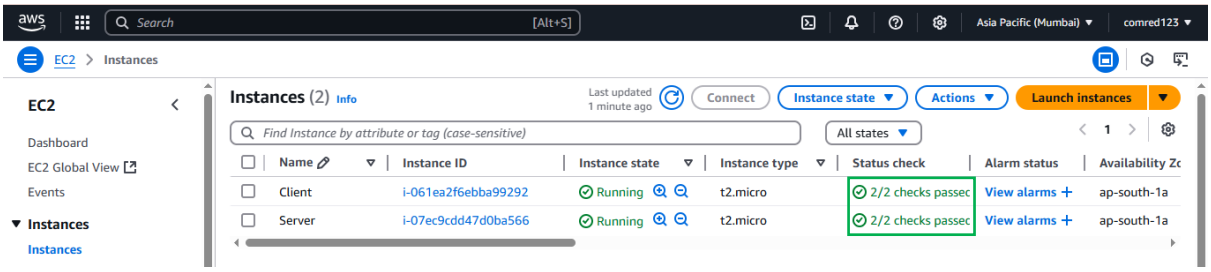


- To create **Server** and **Client** machine, Fill the all necessary parameter

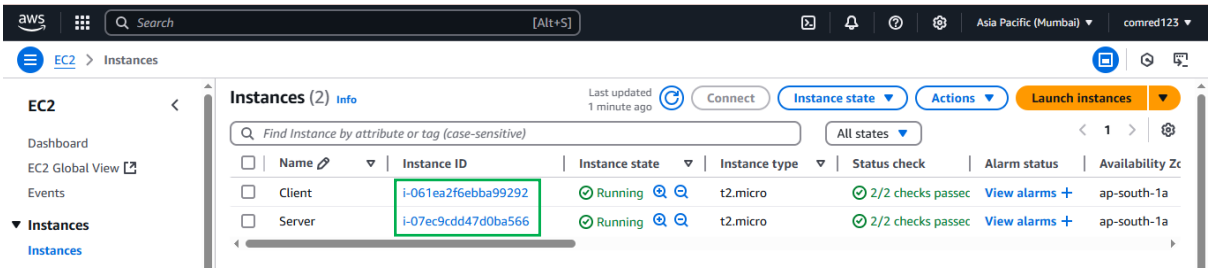




- In the same way create Client Machine
- After click on **Lunch instance,** wait for some time to automatically start machine after showing **2/2 checks passed**



- Click on **Instance ID** to get IP and others details



- **Server** machine will use as a **Controller** and **Client** machine will use as a **Managed** Node.

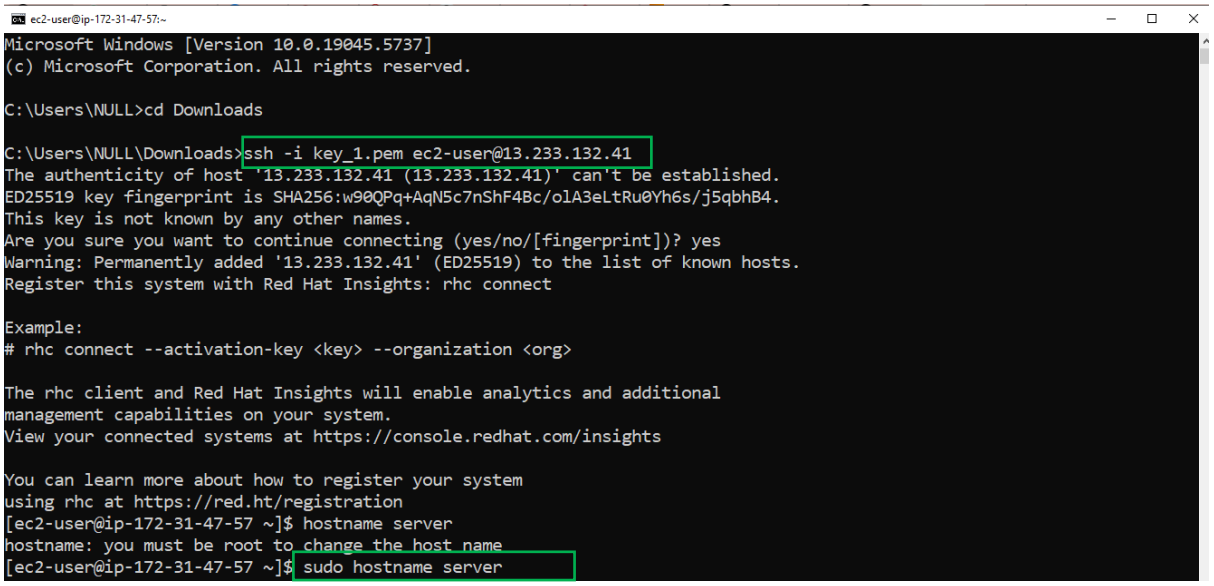
8.1.2 Key Pair Configuration

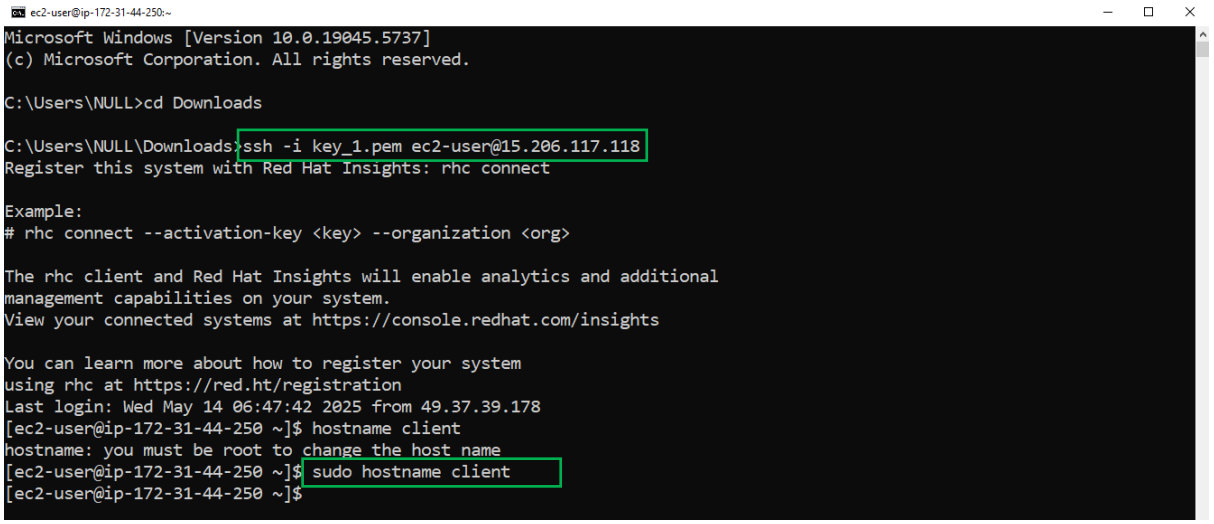
- Create or download a **key pair (.pem)** for secure SSH access.
- Change permission: chmod 400 your-key.pem

8.1.3 Instance Connectivity with CMD

- Open CMD

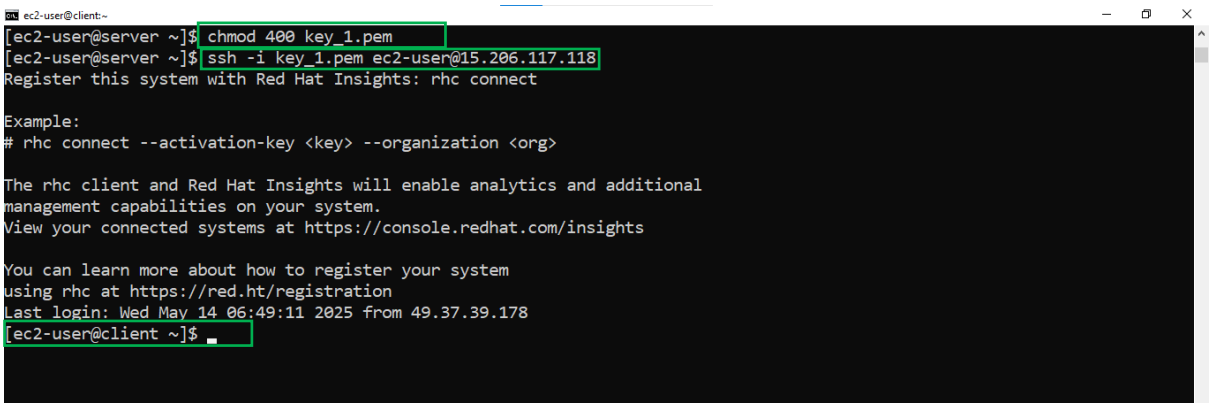
- Give the following command to connect **Server** from AWS ec2 instance and change the hostname to **server**





- Check the **ping**, from **server** to **client.** If the ping is Successful then continue.

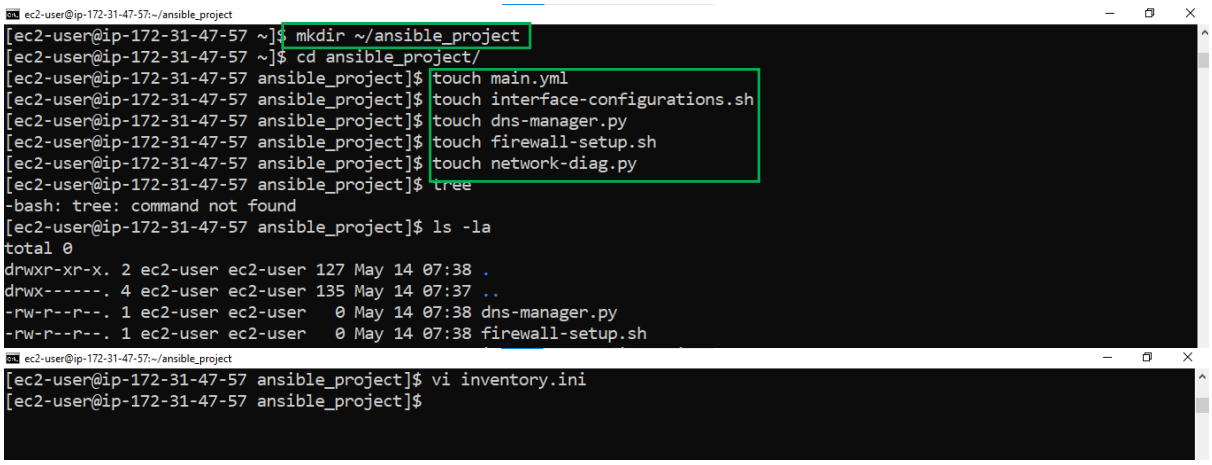
- Now give the **read-only** permission to **key_1.pem** file and check the **SSH** connection from **server** to **client**



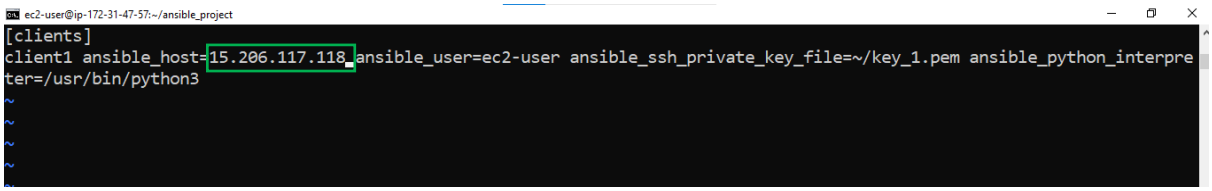
8.2 Create the file structure for Ansible

ansible_project/
- ├── main.yml
- ├── interface-configurations.sh
- ├── dns-manager.py
- ├── firewall-setup.sh
- ├── network-diag.py
- ├── inventory.ini

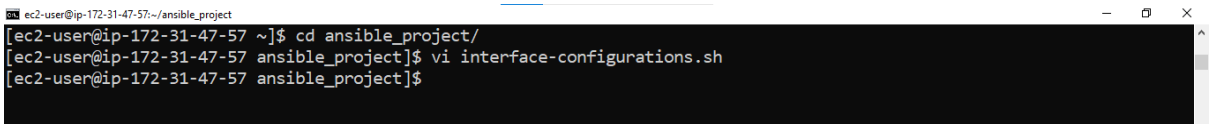
- Create **ansible_project** folder and within this folder create all the above files



- Use the **Client IP** to manage the node



- Now write the **Code** for each above file expect the inventory.ini
- **interface-configurations.sh**



```
ec2-user@ip-172-31-47-57:~/ansible_project
#!/bin/bash

echo "[INFO] Starting dummy interface configuration..."

# Check if dummy module is loaded
if ! lsmod | grep -q dummy; then
    echo "[INFO] Loading dummy kernel module..."
    modprobe dummy
    if [ $? -ne 0 ]; then
        echo "[ERROR] Failed to load dummy module. Are you root?"
        exit 1
    fi
else
    echo "[INFO] Dummy module already loaded."
fi

# Create dummy interface dummy0
ip link add dummy0 type dummy 2>/dev/null
if [ $? -eq 0 ]; then
    echo "[INFO] Dummy interface dummy0 created."
else
    echo "[WARNING] dummy0 already exists or failed to create."
fi

# Assign IP address to dummy0
ip addr add 192.168.100.1/24 dev dummy0 2>/dev/null
if [ $? -eq 0 ]; then
    echo "[INFO] Assigned IP 192.168.100.1/24 to dummy0."
```

```
else
    echo "[WARNING] IP may already be assigned or failed to set."
fi

# Bring interface up
ip link set dummy0 up
if [ $? -eq 0 ]; then
    echo "[SUCCESS] Dummy interface dummy0 is up and ready."
else
    echo "[ERROR] Failed to bring up dummy0."
fi
```

- **firewall-setup.sh**

```
ec2-user@ip-172-31-47-57:~/ansible_project
[ec2-user@ip-172-31-47-57 ansible_project]$ vi firewall-setup.sh
[ec2-user@ip-172-31-47-57 ansible_project]$
```

```
ec2-user@ip-172-31-47-57:~/ansible_project
#!/bin/bash

echo "[INFO] Starting firewall setup for RHEL 9..."

# Check for root
if [ "$EUID" -ne 0 ]; then
  echo "[ERROR] Please run as root."
  exit 1
fi

# Install iptables (legacy) if not present
if ! command -v iptables >/dev/null 2>&1; then
    echo "[INFO] Installing iptables-legacy..."
    dnf install -y iptables-legacy
    if [ $? -ne 0 ]; then
        echo "[ERROR] Failed to install iptables-legacy."
        exit 1
    fi
else
    echo "[INFO] iptables is already installed."
fi

# Switch to legacy iptables if nftables is default
alternatives --set iptables /usr/sbin/iptables-legacy
alternatives --set ip6tables /usr/sbin/ip6tables-legacy
echo "[INFO] Switched to iptables-legacy."
```

ec2-user@ip-172-31-47-57:~/ansible_project

```
iptables -F
iptables -X
echo "[INFO] Flushed all existing rules."

# Set default policies
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
echo "[INFO] Default policies set: INPUT/FORWARD=DROP, OUTPUT=ACCEPT"

# Allow loopback and established sessions
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
echo "[INFO] Allowed loopback and established connections."

# ✅ Allow selected ports
iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # SSH
iptables -A INPUT -p tcp --dport 80 -j ACCEPT   # HTTP
iptables -A INPUT -p tcp --dport 443 -j ACCEPT  # HTTPS
echo "[INFO] Allowed ports: 22 (SSH), 80 (HTTP), 443 (HTTPS)."

# ❌ Deny selected ports
iptables -A INPUT -p tcp --dport 23 -j DROP     # Telnet
iptables -A INPUT -p udp --dport 137 -j DROP    # NetBIOS
iptables -A INPUT -p udp --dport 138 -j DROP
iptables -A INPUT -p udp --dport 139 -j DROP
echo "[INFO] Denied ports: 23 (Telnet), 137-139 (NetBIOS)."
```

- **dns-manager.py**

ec2-user@ip-172-31-47-57:~/ansible_project

```
[ec2-user@ip-172-31-47-57 ansible_project]$ vi dns-manager.py
```

ec2-user@ip-172-31-47-57:~/ansible_project

```
#!/usr/bin/env python3

import os

print("[INFO] Configuring DNS...")

resolv_conf = "/etc/resolv.conf"
dns_servers = ["8.8.8.8", "1.1.1.1"]

try:
    with open(resolv_conf, "w") as f:
        for dns in dns_servers:
            f.write(f"nameserver {dns}\n")
    print(f"[SUCCESS] DNS servers updated: {', '.join(dns_servers)}")
except Exception as e:
    print(f"[ERROR] Failed to write to {resolv_conf}: {e}")
```

- **network-diag.py**

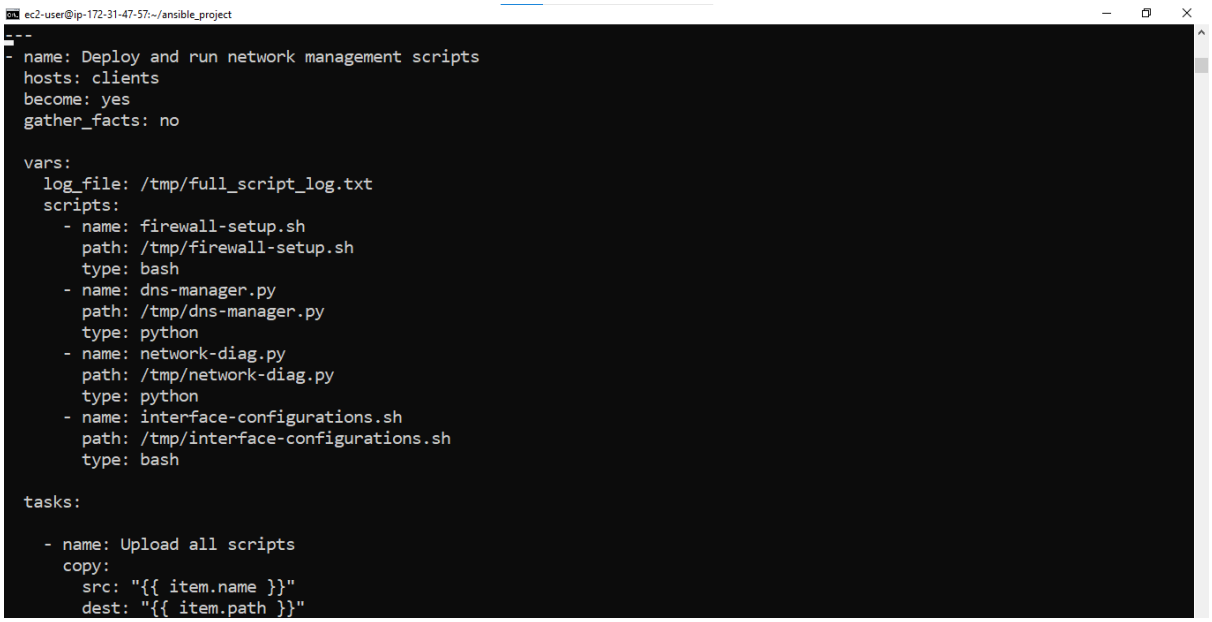
ec2-user@ip-172-31-47-57:~/ansible_project

ec2-user@ip-172-31-47-57:~/ansible_project

```
#!/usr/bin/env python3

import subprocess

print("[INFO] Starting network diagnostics...")

# Example diagnostics
commands = [
    ["ping", "-c", "2", "8.8.8.8"],
    ["ip", "a"],
    ["ip", "r"]
]

for cmd in commands:
    try:
        output = subprocess.check_output(cmd, stderr=subprocess.STDOUT).decode()
        print(f"[COMMAND] {' '.join(cmd)}")
        print(output)
    except subprocess.CalledProcessError as e:
        print(f"[ERROR] {' '.join(cmd)} failed:\n{e.output.decode()}")
```

*To save all the code in **vi** editor use **Esc** and **:wq**

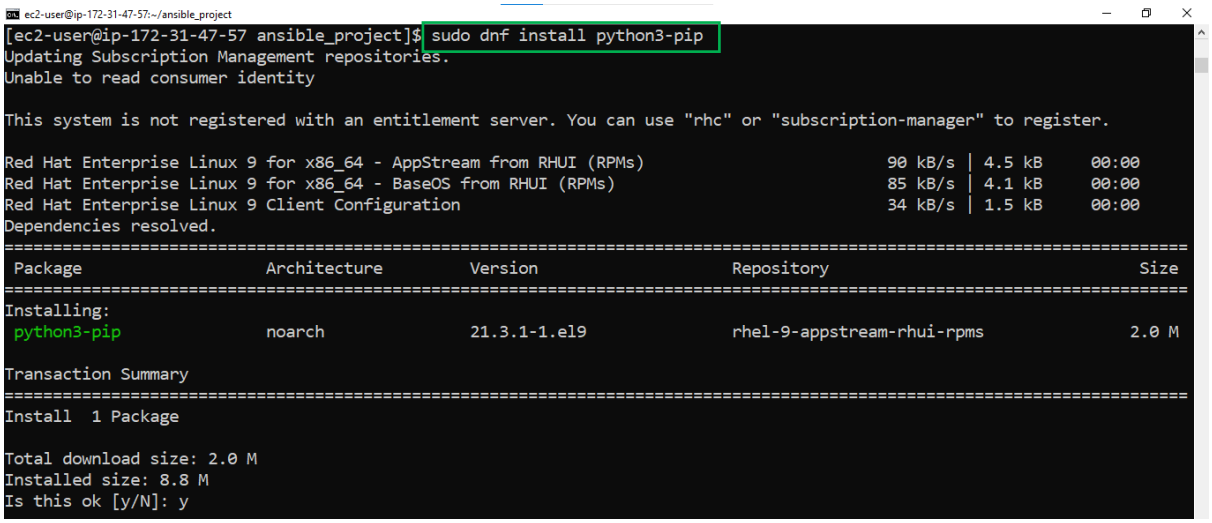
- **main.yml**



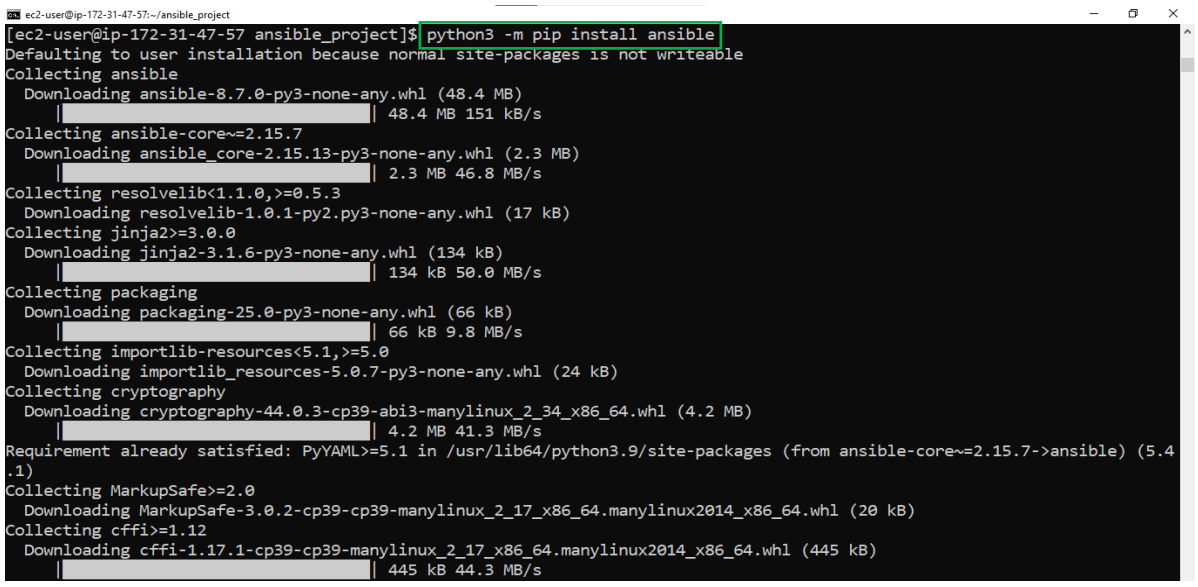

```
  - name: Upload all scripts
    copy:
      src: "{{ item.name }}"
      dest: "{{ item.path }}"
      mode: '0755'
    loop: "{{ scripts }}"

  - name: Run all scripts and log output
    shell: |
      echo "===== START: {{ item.name }} =====" >> {{ log_file }}
      echo "Timestamp: $(date)" >> {{ log_file }}
      {{ 'bash' if item.type == 'bash' else 'python3' }} {{ item.path }} >> {{ log_file }} 2>&1
      echo "===== END: {{ item.name }} =====" >> {{ log_file }}
      echo "" >> {{ log_file }}
    loop: "{{ scripts }}"
    ignore_errors: yes

  - name: Final timestamp in log
    lineinfile:
      path: "{{ log_file }}"
      line: "All scripts executed at {{ lookup('pipe', 'date') }}"
```

8.3 Install Ansible to run ansible-playbook

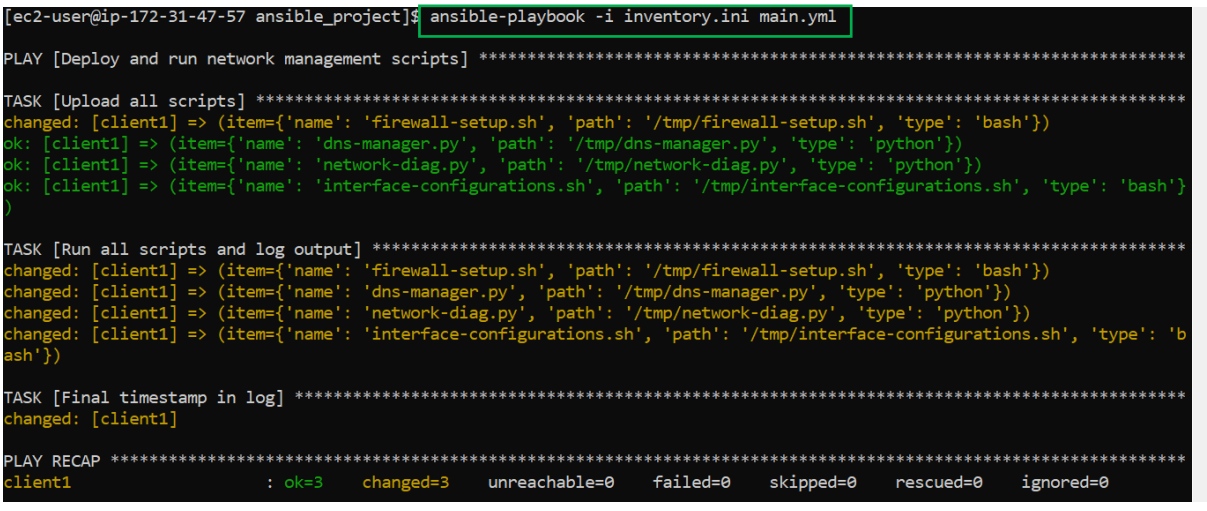
- First install dependencies



- Now install ansible



- Run the **Ansible-Palybook** using following command



8.4 Logging and Output Handling

- After the **cat /tmp/full_script_log.txt**, we can see the **file logs**

```
===== START: firewall-setup.sh =====
Timestamp: Wed May 14 08:37:56 AM UTC 2025
Installing iptables...
Updating Subscription Management repositories.
Unable to read consumer identity

Transaction Summary
================================================================================
Install  3 Packages

Total download size: 322 k
Installed size: 805 k
Downloading Packages:
(1/3): iptables-nft-1.8.10-11.el9_5.x86_64.rpm  4.0 MB/s | 209 kB     00:00
(2/3): iptables-nft-services-1.8.10-11.el9_5.no 452 kB/s |  24 kB     00:00
(3/3): libnftnl-1.2.6-4.el9_4.x86_64.rpm        1.5 MB/s |  89 kB     00:00
--------------------------------------------------------------------------------
Total                                           2.9 MB/s | 322 kB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : libnftnl-1.2.6-4.el9_4.x86_64                          1/3
  Installing       : iptables-nft-1.8.10-11.el9_5.x86_64                    2/3
  Running scriptlet: iptables-nft-1.8.10-11.el9_5.x86_64                    2/3
  Installing       : iptables-nft-services-1.8.10-11.el9_5.noarch           3/3
  Running scriptlet: iptables-nft-services-1.8.10-11.el9_5.noarch           3/3
  Verifying        : iptables-nft-services-1.8.10-11.el9_5.noarch           1/3
  Verifying        : libnftnl-1.2.6-4.el9_4.x86_64                          2/3
  Verifying        : iptables-nft-1.8.10-11.el9_5.x86_64                    3/3
Installed products updated.
[+] Disabling firewalld...
Failed to stop firewalld.service: Unit firewalld.service not loaded.
Failed to disable unit: Unit file firewalld.service does not exist.
[+] Enabling iptables...
[+] Flushing existing iptables rules...
[+] Allowing ports: 22, 80, 443
[+] Blocking ports: 23, 8080
[+] Dropping all other traffic
[+] Saving rules and restarting iptables
iptables: Saving firewall rules to /etc/sysconfig/iptables: [  OK  ]
[+] Current iptables rules:
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0           ctstate RELATED,ESTABLISHED
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:80
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:443
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:23
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:8080
DROP       all  --  0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
===== END: firewall-setup.sh =====
```

```
===== START: dns-manager.py =====
Timestamp: Wed May 14 08:38:00 AM UTC 2025
[INFO] Configuring DNS...
[SUCCESS] DNS servers updated: 8.8.8.8, 1.1.1.1
===== END: dns-manager.py =====
```

```
===== START: network-diag.py =====
Timestamp: Wed May 14 08:38:01 AM UTC 2025
[INFO] Starting network diagnostics...
[COMMAND] ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=2.10 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=1.83 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.829/1.966/2.104/0.137 ms

[COMMAND] ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:96:e0:bd:fb:4f brd ff:ff:ff:ff:ff:ff
    altname enX0
    inet 172.31.44.250/20 brd 172.31.47.255 scope global dynamic noprefixroute eth0
       valid_lft 2298sec preferred_lft 2298sec
    inet6 fe80::96:e0ff:febd:fb4f/64 scope link
       valid_lft forever preferred_lft forever
```

```
[COMMAND] ip r
default via 172.31.32.1 dev eth0 proto dhcp src 172.31.44.250 metric 100
172.31.32.0/20 dev eth0 proto kernel scope link src 172.31.44.250 metric 100

===== END: network-diag.py =====
```

```
===== START: interface-configurations.sh =====
Timestamp: Wed May 14 08:38:03 AM UTC 2025
[INFO] Starting dummy interface configuration...
[INFO] Loading dummy kernel module...
[INFO] Dummy interface dummy0 created.
[INFO] Assigned IP 192.168.100.1/24 to dummy0.
[SUCCESS] Dummy interface dummy0 is up and ready.
===== END: interface-configurations.sh =====
```

9. RESULTS & ANALYSIS :

The successful implementation of the project was validated by executing multiple Ansible playbooks across AWS-based managed nodes. Each playbook delivered consistent, repeatable results, demonstrating the power of infrastructure automation. The outcomes from interface configuration, DNS setup, firewall enforcement, and network diagnostics were verified through terminal output and log files.

9.1 Interface Configuration Output

- The playbook successfully executed the interface-configurations.sh script on all managed nodes.
- A **dummy interface dummy0** was created and assigned the IP address 192.168.100.1/24.
- Command used for verification:

```
ip a | grep dummy0
```

- Output confirmed the interface was up and assigned the correct IP.
- Re-running the playbook produced no error, demonstrating **idempotency**.

9.2 DNS Configuration Output

- The dns-manager.py script updated /etc/resolv.conf with DNS servers 8.8.8.8 and 1.1.1.1.
- Validation was done by:

```
cat /etc/resolv.conf
```

- The output matched the expected DNS configuration.
- DNS resolution was tested using:

```
ping google.com
```

- Name resolution succeeded, indicating correct DNS setup.

9.3 Firewall Rules and Port Access Test

- The firewall-setup.sh script was used to apply **iptables rules**:
  - **Allowed ports**: 22 (SSH), 80 (HTTP), 443 (HTTPS)
  - **Denied ports**: 23 (Telnet), 137-139 (NetBIOS)
- Verification was performed using:

```
sudo iptables -L -n -v
```

- Manual testing:
  - SSH and HTTP access worked correctly.
  - Telnet and NetBIOS ports were blocked using nmap from another system:

```
nmap -p 23,137 172.31.xx.xx
```

- Output showed closed ports for denied services, validating firewall effectiveness.

9.4 Network Diagnostics Output

- The network-diag.py script executed:
  - ping 8.8.8.8
  - ip a

    - ip r
- Output included:
    - Round-trip time from ping
    - Active IP interfaces
    - Route tables
- These were logged and reviewed to verify network health.

9.5 Logging Verification

- A centralized log file (ansible_run.log) captured:
    - Task start and end times
    - Script outputs
    - Any warnings or errors
- Logs were used to:
    - Verify success of each task
    - Debug failed executions (if any)
    - Provide documentation for project evaluation

9.6 Error Handling and Troubleshooting

- **Firewall setup failed** initially on one node due to iptables-legacy not being installed. This was resolved by ensuring dnf install iptables-legacy was included in the playbook.
- **SSH key errors** occurred during first-time setup, resolved by manually copying keys using ssh-copy-id.
- **Idempotent behavior** was confirmed across all playbooks—no duplication or disruption occurred on re-runs.

10. CONCLUSION :

This project demonstrates the practical and effective use of **Ansible** as a powerful automation tool for managing and configuring Linux-based systems in a scalable and consistent manner. By automating four critical system administration tasks—**interface configuration**, **DNS setup**, **firewall management**, and **network diagnostics**—the project has successfully shown how Infrastructure as Code (IaC) can replace manual, error-prone processes with reliable, repeatable, and efficient automation.

Implementing the project on **AWS EC2 instances** added a cloud-native dimension to the work, reflecting real-world enterprise environments where scalability, remote access, and automation are essential. The Ansible playbooks, developed in YAML and executed from a central control node, streamlined operations across multiple remote systems without requiring any agent installation on the client machines.

Key benefits observed during implementation included:

- **Significant reduction in configuration time**
- **Improved accuracy and consistency** of system settings
- **Enhanced security posture** through enforced firewall policies
- **Effective use of centralized logging** for troubleshooting and verification

Additionally, the principle of **idempotency** ensured that all tasks could be safely re-executed without disrupting system state, which is crucial for maintaining configuration drift in dynamic IT environments.

This project not only enhances system administration skills but also introduces students to modern DevOps practices and tools used in production-level infrastructure management. It serves as a foundation for more advanced automation projects, such as continuous deployment pipelines, cloud orchestration, and compliance enforcement.

11. FUTURE SCOPE:

The current project lays a solid foundation for automating essential system configuration tasks using Ansible. However, there are several ways in which this project can be expanded and enhanced to meet the demands of larger, more complex, and more dynamic IT infrastructures. The following points highlight the potential future developments and applications of this work:

11.1 Integration with CI/CD Pipelines

- This automation framework can be extended to work as part of **Continuous Integration/Continuous Deployment (CI/CD)** pipelines using tools like **Jenkins**, **GitLab CI**, or **GitHub Actions**.
- Ansible playbooks can be triggered automatically after code commits to configure infrastructure or deploy applications.

11.2 Dynamic Inventory with Cloud Providers

- Instead of using a static inventory file, the project can implement **dynamic inventory** by integrating with AWS EC2 APIs.
- This allows Ansible to automatically detect and manage EC2 instances, making it suitable for highly dynamic environments such as auto-scaling groups.

11.3 Security Hardening and Compliance Automation

- The firewall configuration can be extended to include **SELinux policies**, **audit logging**, **automatic patching**, and **compliance enforcement** (e.g., CIS benchmarks).
- Integration with tools like **OpenSCAP** or **Ansible Security Collections** can automate hardening procedures.

11.4 Web-based GUI for Playbook Execution

- A front-end can be developed using tools like **AWX** (Ansible Tower's open-source version) to provide a **user-friendly interface** for launching playbooks, monitoring jobs, and managing inventories.
- This makes the system accessible to non-technical users or IT helpdesk teams.

11.5 Cross-Platform and Multi-Cloud Automation

- Extend automation beyond RHEL-based systems to include **Windows servers**, **Debian-based distributions**, and **containerized environments**.
- Incorporate **multi-cloud support** (AWS, Azure, GCP) to manage hybrid infrastructure using a single Ansible control node.

12. LIMITATIONS :

While this project effectively demonstrates the power of Ansible for automating network configurations, DNS management, firewall setup, and diagnostics, there are several **limitations** that constrain its scope, scalability, and adaptability in real-world production environments. These limitations are outlined below:

12.1 Limited to Linux Environments

- The project was designed and tested only on **RHEL-based Linux distributions** (RHEL 8 and 9).
- **Windows or macOS systems are not supported**, restricting the project's applicability to heterogeneous environments.

12.2 Static Inventory Management

- The use of a **static inventory file** requires manual updates to track managed nodes.
- This approach is not scalable in dynamic cloud environments like AWS where instances frequently change, unlike **dynamic inventories** that use EC2 metadata.

12.3 Basic Logging Mechanism

- Logging is implemented using Ansible's default logging configuration and inline script outputs.
- It lacks **centralized log aggregation**, structured logging (e.g., JSON format), or integration with monitoring tools like **ELK Stack**, **Grafana**, or **Prometheus**.

12.4 No GUI or Dashboard for Execution

- All automation tasks are executed via the command line, which may be challenging for non-technical users.
- The project does not include a **graphical interface** such as **AWX** or **Ansible Tower**, which could simplify execution and monitoring.

12.5 Non-Persistent Firewall Rules

- The iptables rules configured during execution are not made **persistent across reboots** unless saved manually using tools like iptables-save.
- This may result in loss of firewall configurations upon system restart, posing a potential security risk.

12.6 Limited Error Handling and Validation

- The scripts and playbooks assume successful task execution with minimal conditional checks or validations.
- There is **no advanced error recovery**, retry mechanisms, or notifications (e.g., email/Slack alerts) in case of failure.

13. REFERENCES:

- Red Hat. (n.d.). *Red Hat Enterprise Linux 9 Documentation*. https://access.redhat.com/documentation/en-us/
- Ansible Documentation. (n.d.). *Official Ansible Documentation*. https://docs.ansible.com/
- Forsyth, M. (2019). *Automate the Boring Stuff with Python (2nd ed.)*. No Starch Press. https://automatetheboringstuff.com/
- White, R. (2021). *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media. https://www.oreilly.com/library/view/infrastructure-as-code/9781098114665/
- Ansible. (n.d.). *Ansible Use Cases: IT Automation for Real-World Problems*. https://www.ansible.com/use-cases
- Configr Technologies. (2024, March 1). *Network Automation with Python and Ansible – Streamlining Configuration, Provisioning, and Management*. https://medium.com
- Oswalt, M. (2015, August 2). *Network Automation using Ansible and Python* [Video]. https://www.youtube.com. *Next Day Video*.